IPA

Week 11 – Terraform: IaC concept and Introduction to Terraform

School of Information Technology, KMITL
2025

This presentation are created

Based on Material from

- Ned Bellavance : Terraform – Getting Started
- Terraform in Dept

What is Infrastructure as Code (IaC)

Provisioning infrastructure through software to achieve CONSISTENT and PREDICTABLE environments

What is Infrastructure as Code (IaC)

- Defined in code
- Stored in source control
- Declarative or Imperative
- Idempotent and consistent

Types of IaC Tools

- Ad hoc script
- Configuration management tools
- Server templating tools
- Orchestration tools
- Provisioning tools

Ad hoc script

- Bash scripts
  - apt-get update
  - apt-get install
  - …
-
- Each one has his own style to complete the same task
- Work well with small / one-off task

Configuration management tools

- Chef, puppet, ansible
- Many advantages over ad hoc scripts
- Consistent structure
- Idempotence (explain later)
- Distribution to large number of hosts

Server templating tools

- Packer, vagrant
- Create an image of a server that captures a fully self-contained snapshot of OS and all relevant detail

Orchrestration tools

- Tools to manage different components (deploy, update, monitor, scaling, HA…)
- Mesos, Amazon ECS,..

Provisioning tools

- Terraform, CloudFormation

Declarative or imperative

#Make me a cup of coffee

get coffee beans

get water

get filter paper

get dripper

ground coffee beans

boil water

put coffee in dripper

put water through coffee beans



This Photo by Unknown Author is licensed under CC BY-NC-ND

Declarative or imperative

#Make me a cup of coffee

Filter “coffee-dripper” {
  ingredients = [
    “coffee beans”, “water”, “filter paper”
  ]
}


This Photo by Unknown Author is licensed under CC BY-NC-ND

Declarative or imperative

- Some IaC tools are Imperative
  - Many configuration management tools are Imperative
- Some IaC tools are Declarative
  - Terraform is based on Declarative

Advantage of Declarative over Imperative

- Declarative language make it easier to define relationships between components without having to explicitly define it
- Create VPC and then using VPC ID to create Subnet

Problem of Declarative Language

- Circular dependency
- Keep waiting on the other, none of resources can successfully launch

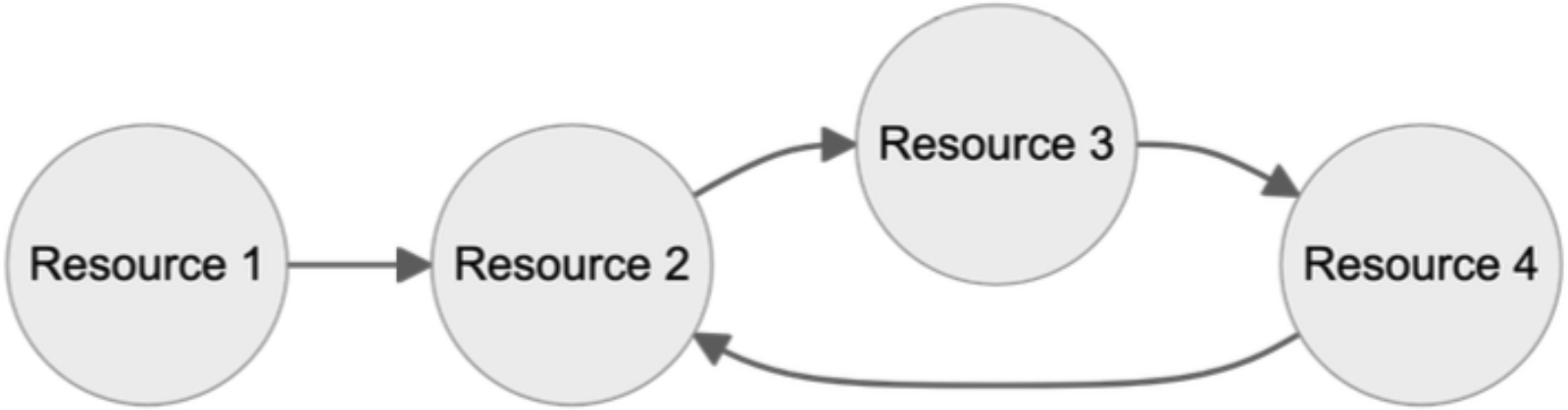


Figure 1.4 An example of a circular dependency in which resources are dependent on one another.

Idempotent



Make me coffee



Here's your coffee



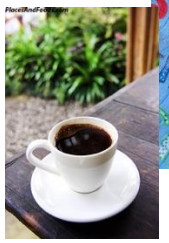
This Photo by Unknown Author is licensed under CC BY-NC-ND

This Photo by Unknown Author is licensed under CC BY-NC-ND

Idempotent

Make me coffee





This Photo by Unknown Author is licensed under CC BY-NC-ND

You already have coffee



This Photo by Unknown Author is licensed under CC BY-NC-ND

IaC Benefits

- Automated deployment
- Consistent environments
- Repeatable process
- Reusable components
- Documented architecture

CI/CD Platform for Terraform

- HCP Terraform : https://developer.hashicorp.com/terraform/cloud-docs
- Scalr : https://scalr.com/
- Spacelift : https://spacelift.io

Terraform Architecture

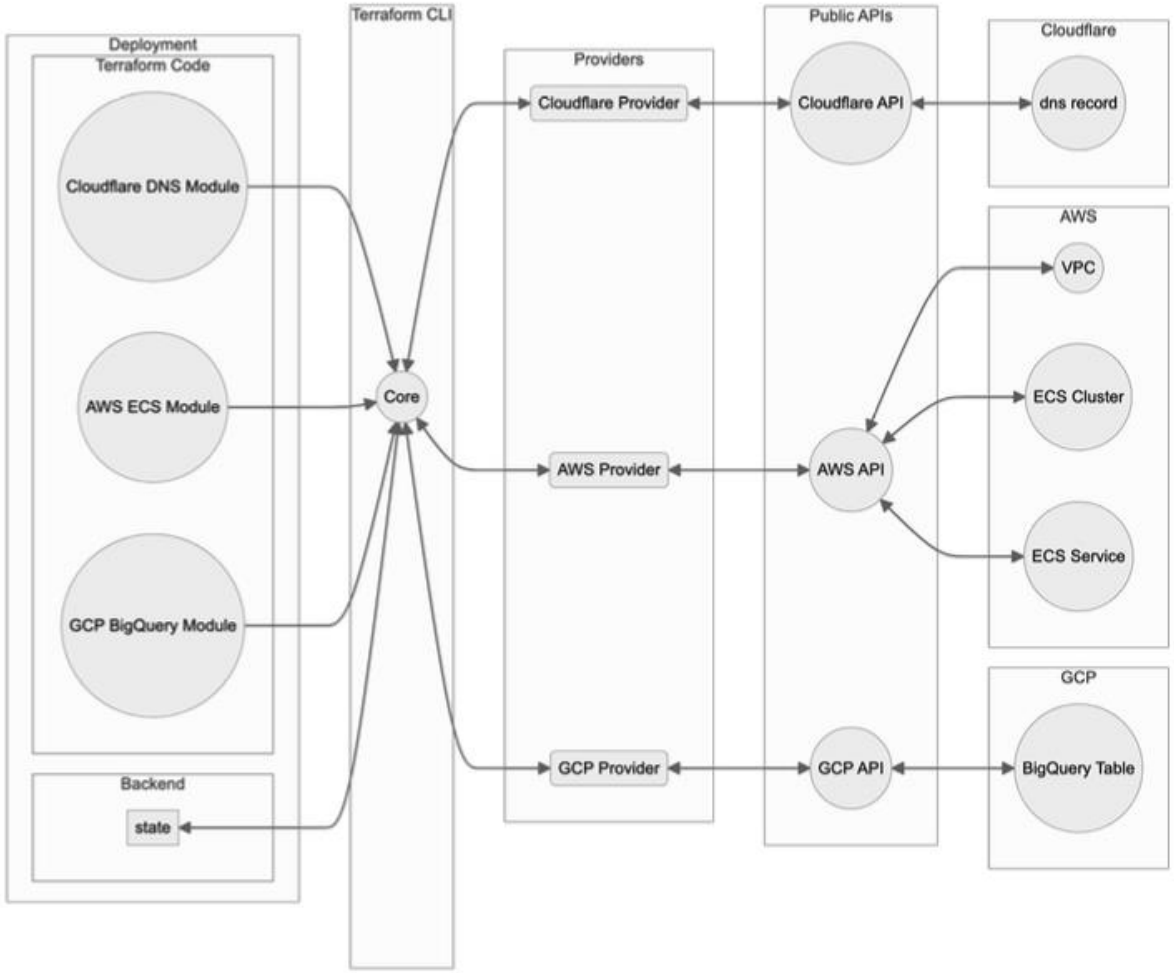


Figure 1.1 Terraform has multiple levels of abstraction that allow developers to get things done without worrying about vendor-specific minutiae.

How Terraform works

- Terraform is an opensource tool created by HachiCorp
- Written in GO language
- We can install terraform binary to deploy infrastructure from our machine
- Terraform binary makes API calls to one or more provider on our behalf (such as AWS, Azure, GCP, DigitalOcean,..)
- We can instruct the API calls in Terraform configurations

Do Terraform provide transparent portability between Cloud provider?

- No

- Because the cloud providers do not offer the same types of infrastructure
- One functionality in one cloud provider may not exist in others
- Terraform allow you to use same language, toolsets for all providers

Terraform CLI

- Primary way to run and managed Terraform projects
- CLI is used to create new infrastructure, update, destroy and apply changes

Terraform Providers

- Provider is a plugin for terraform to supplies a collection of Data sources, Resources and Function for managing infrastructure with a specific vendors
- Somes are managed directly by HashiCorp and by their specific vendors

Terraform Backends

- Can customize where a Workspace stores its state files
- Local Backend is a default workspace
- Cannot share state files with other
- For testing but cannot scale to teams
- Can move to other locations in cloud storage such as S3

Terraform Workspaces

- Representing a deployment of a Terraform with a specific backend and set of input variables

Terraform: Getting start

- The core of Terraform's functionality is provided by Terraform binary
- Let begin with 3 main commands
  - plan – show what changes Terraform will make to the infrastructure
  - Apply – apply changes to the infrastructure
  - Destroy – destroy infra. built with Terraform

Terraform: Getting start

- Terraform configuration files
- When Terraform runs inside a directory, it will load Terraform configuration files, any non-configuration files are ignore
- Then, Terraform constructs a DAG (Directed Acyclic Graph) -> its nodes are resources -> a host, subnet, storage and the edges are the relationship
- The graph determins the relationship and ensure Terraform build the infrastructure in the right order
  - VPC before EC2 ..

Terraform: Getting start

- Terraform configuration → .tf or .tf.json
- .tf is easy for human to read
- .tf.json is pure JSON -> mean for machine interaction

Terraform: Getting start

- Terraform configuration components
- See the example files
- There are multiple components
  - Providers
  - Data
  - Resource

Terraform: Providers

- Connect Terraform to the infrastructure that we want to manage
- For example, AWS, GCP, Azure, ..
- Providers are no shipped with Terraform
- Must download the providers that you need
- Using command: terraform init

Terraform: Resources

- Represent the infrastructure components that we want to create/manage

Terraform: Built the infrastructure

- Create execution plan
- We can run execution plans in one of two modes: Plan (display the proposed changes to make) and Apply (Apply the change)
- Plan is for checking that we are going to make the right changes
- Best practice is to running a plan before you apply any changes

Terraform state

- After creating the resources, Terraform has saved the current state of the infrastructure into a file called terraform.tfstate in the work directory

Terraform Destroy

- Destroy the infrastructure that we built by Terraform configuration

Terraform Documents

- Docs overview | hashicorp/aws | Terraform Registry
- https://registry.terraform.io/providers/hashicorp/aws/latest/docs#getting-the-account-id

Assignment: Create these resources using Terraform

- VPC specification
  - Name: testVPC
  - CIDR block: 10.0.0.0/16
  - Create Public subnet with Name “Public1” using 10.0.1.0/24
    - This subnet must be created in AZ us-east-1b

Assignment: Create these resources using Terraform

- Create Security Group
  - Name: AllowSSHandWeb
  - Allow incoming SSH and HTTP traffic to EC2 Instance
- Using vockey as the key pair
- EC2 Specification
  - Using Amazon Linux2 AMI
  - T2.micro
  - Deploy in Public1
  - Use EBS 8 GB General purpose SSD
  - TAG: Key = Name, Value = tfTest
  - Using Security Group name: AllowSSHandWeb (allow ssh and HTTP from all source ip addresses)
  - Using vockey as a key pair